

HOWTO — capturing real SNES screenshots from MAME and bsnes-jg, headless

What: get a true, emulator-*rendered* PNG of the SNES screen (the PPU's actual output) out of both emulators we run, with no window, no X session, no GPU — suitable for CI and for the `dev/run.sh` Docker harness. Driven end-to-end by `dev/mandel-shot.sh` (`dev/run.sh mandel-shot`).

Why it was non-obvious: the project's emulator harness is built for *memory* assertions (read WRAM, compare bytes — `dev/smoke.lua` , `dev/jgxcheck.cpp`). Getting a *picture* out is a different problem, and the two cores solve it in opposite ways. This was figured out building the #321 Mandelbrot demo (Track 2); the findings are recorded here so the next person doesn't re-derive them.

TL;DR

Core	Mechanism	Headless?	Output
bsnes-jg	Software renderer fills a user buffer every frame; <code>dev/jgxcheck.cpp</code> reads it and writes a PNG	natively (no X at all)	256×224 (downsampled from native 512×240)
MAME	<code>manager.machine.video:snapshot()</code> from a Lua autoboot script, run under Xvfb	needs a virtual X display	512×225 (MAME's native snes snapshot)

Both are exercised by `dev/run.sh mandel-shot` → `build/mandel-jg.png` and `build/mandel-mame.png` .

bsnes-jg — dump the software framebuffer (the easy, fully-headless path)

bsnes-jg renders entirely in software into a caller-supplied buffer — there is no display surface to capture from, you just *read the buffer*. `dev/jgxcheck.cpp` already allocates that buffer (`vbuf`) and registers it via `Bsnes::setVideoSpec` , because the harness needs the emulator running anyway. So capturing a frame is: run N frames, then write `vbuf` to a PNG.

Key facts (from `vendor/bsnes-jg/src/ppu.cpp` , `PPU::refresh` + `PPU::genPalette`):

- **Callback:** `videoFrame(userPtr, width, height, pitch)` — note the first arg is the *user pointer* (`spec.ptr` , null in our harness), **not** the pixels. The pixels are in the buffer you passed to `setBuffer` / `setVideoSpec` . Read that buffer, not the callback arg.
- **Geometry:** native frame is **512×240** (NTSC, non-interlace), `pitch == 512` in **pixels** (not bytes). In lores 256-px modes each pixel is doubled to fill 512 — take every other column to recover the true 256.
- **Pixel format:** `0x00RRGGBB` (red in the high byte of the low 24 bits). This is the gotcha: the `lightTable` output is `ab<<16 | ag<<8 | ar` , indexed by `(r<<10)|(g<<5)|b` . SNES CGRAM is **BGR555** (red in the low bits), so the raw-CGRAM lookup lands the *red* intensity in bits 16-23. Extract `R=(px>>16)&0xFF` , `G=(px>>8)&0xFF` , `B=px&0xFF` . (A solid-green test screen can't catch a R/B swap — use a red or blue test, or a known palette.)

`jgxccheck` writes the PNG with the shared `tools/png_write.h` encoder. Optional knobs (env):
`JGX_FULLFRAME=1` dumps the raw 512×240 (no crop/downsample — use to debug offset); `JGX_YOFF=N` skips N top scanlines (overscan); `JGX_VRAM=1` prints a VRAM tilemap/char hex dump (debugging the blit).

```
# 7th arg = output PNG (added for this):
jgxccheck <rom.sfc> <bsnes-Database/> <wram_off_hex> <len> <want_hex> <frames> <out.png>
```

MAME — snapshot under Xvfb (the renderer needs a real surface)

MAME *can* write a PNG headless — `manager.machine.video:snapshot()` from a Lua autoboot script drops a file in `-snapshot_directory` . **But with `-video none` or `SDL offscreen / soft / accel` the PPU output never reaches the snapshot surface, so the PNG is all-black** (confirmed across all those video backends). The fix is to give MAME a real (virtual) X display with **Xvfb**:

```
SHOT_ADDR=0x7E0580 SHOT_WANT=0x9103 \
xvfb-run -a mame snes -cart rom.sfc -rompath dev/roms \
  -autoboot_script dev/mandel-shot.lua -skip_gameinfo \
  -snapshot_directory build/.snap -sound none -nothrottle -seconds_to_run 30
# -> build/.snap/snes/0000.png (512x225 RGB)
```

- The dev image carries Xvfb in its own Dockerfile layer (after toolchain/mame, so it never re-triggers them). `xvfb-run -a` picks a free display number automatically.
- **Do NOT pass `-video none`** here — that's what kills the snapshot. Let MAME use its default renderer against the Xvfb display.

- The autoboot Lua (`dev/mandel-shot.lua`) waits enough frames for the program to finish drawing (the Mandelbrot fill is ~900 frames; snapshot at ~1400), calls `video:snapshot()` , and can assert WRAM in the same pass so one run gives both the picture and the memory proof.
- MAME's snes snapshot is **512×225** (double-width, like bsnes's native 512). Downsample by 2 horizontally if you want to compare pixel-for-pixel against the 256-wide bsnes/host images.

Proving the screenshot is the *verified* image

A screenshot alone could be a render artifact. The demo ties it to the differential gate: the on-console ROM (`mandel-display.c`) leaves the escape-buffer CRC in `corpus_result` (WRAM), and both capture paths assert it equals the **host** renderer's CRC over the same grid (`tools/mandel-render` , same `mandel.h`). So the pixels on screen are provably the same bytes the host computed and the gate (`k_mandel`) verified — `host == default == +mos-a16` , on both emulators, in memory *and* on screen.

Gotchas worth remembering

- **Initialise the PPU.** SNES power-on leaves the PPU control registers indeterminate, and **bsnes-jg randomises them** — so a ROM that sets only the registers it "uses" renders *nondeterministically* (random mosaic / window mask / colour-math enable corrupts the picture differently each boot; VRAM is identical but the frame differs run-to-run). Always force-blank and zero the control registers first (`snes_ppu_reset_blank()` in `platforms/snes/snes.h`). This was the single biggest time sink here.
- **bsnes is deterministic only once you've removed the power-on dependence** — verify by capturing the same ROM 3× and diffing the PNG SHA.
- **Stored-DEFLATE PNGs are constant size** regardless of content (`png_write.h` doesn't compress), so don't infer "blank vs full" from file size — decode or compare SHAs / mean pixel.
- **R/B channel order** differs from what the `lightTable` source literally reads (see above) — test with a non-green color.